

Zou_lab_control.neutral_atom 轻量教程

Jupyter-first camera / sitemap / threshold / detection session

中性原子实验控制最小闭环、frontend 画图接口和后续扩展边界

2026-06-01

Contents

1	这版要解决什么	1
1.1	先跑通一条真实实验会用的线	1
1.2	核心原则	1
2	模块和对象关系	3
2.1	新的 package tree	3
2.2	依赖方向	4
2.3	为什么 camera 是 device	4
2.4	为什么 readout 是一个 subsystem	5
2.5	session 的职责边界	5
2.6	文件边界	6
2.7	device contract	7
2.8	session 里面有什么	7
2.9	standalone 算法入口	7
2.10	Result object 和 summary()	8
3	Frontend 更新	9
3.1	统一 plot 契约	9
3.2	标题	9
3.3	Pulse sequence 图	9
3.4	2D 图保持 square 公开接口	9
4	Device 配置与 virtual camera	11
4.1	最小 device graph	11
4.2	qCMOS-like simulation	11
5	Timing、preflight 和 Verilog	13
5.1	PulseSequence 的作用	13
5.2	preflight 检查	13
5.3	Verilog 生成	13
6	Camera capture	14
6.1	推荐调用	14

6.2	CaptureResult	14
7	Sitemap calibration	16
7.1	推荐调用.	16
7.2	常见错误.	16
8	Threshold calibration	18
8.1	推荐调用.	18
8.2	为什么不能用 simulator truth 标定.	19
9	Detect occupancy.	20
9.1	推荐调用.	20
9.2	图像设计.	21
10	Detection-time fidelity scan	22
10.1	推荐调用.	22
10.2	停止 live scan: 一个对外 stop	22
10.3	用 scan.data_figure 做 decay fit.	22
10.4	reference exposure 的角色	23
11	保存输出	24
11.1	calibration、status、Verilog	24
12	以后加功能应该放哪里.	25
12.1	加真实 camera.	25
12.2	加新的图像算法.	25
12.3	加新的实验动作.	25
12.4	加新的硬件角色.	25
12.5	加 GUI	26
13	下一步扩展计划	27
13.1	真实 qCMOS	27
13.2	sequencer 和 address-switch.	27
13.3	rearrangement 和 GUI.	27

1

这版要解决什么

1.1 先跑通一条真实实验会用的线

当前 `Zou_lab_control.neutral_atom` 只承诺一条最小闭环:

Experiment loop

```
connect devices
-> configure pulse sequence
-> capture camera image
-> calibrate sitemap
-> calibrate threshold
-> detect occupancy
-> scan detection time vs fidelity
```

这不是把 Jupyter 当命令行。session 仍然是实验配置的容器，device、timing、analysis、plot、result object 的边界先放对。之后要加真实 qCMOS、FPGA、AOD rearrangement 或 GUI panel，都应该沿着这些边界补实现，而不是把 camera handle、Matplotlib artist、threshold array、Verilog state 又混进一个 cell。

1.2 核心原则

- device 只负责硬件动作，例如 camera acquire、sequencer prepare/fire。
- `operations` 函数只吃 array 和 calibration，可以 standalone 离线运行。
- subsystem 负责把当前 `exp` 的 device、timing、defaults、calibration 串成实验动作。
- frontend 只画图 and 刷新共享 array，不知道硬件细节。
- result object 保存 raw data、summary 和 plot handle，方便 notebook 交互与复盘。

为什么不把所有函数都放在 `exp` 上

`exp.capture()` 很短，但长期看会让 session 变成上帝对象。现在推荐 `exp.camera.capture()` 和 `exp.readout.sitemap()/thresholds()/detect()/detection_time().camera`

是 device 本体; readout 是一个有机 subsystem, 因为 sitemap、threshold、detect 和 readout fidelity calibration 都共享同一份 `TrapCalibration` 状态和同一套 camera-readout 假设。

2

模块和对象关系

2.1 新的 package tree

现在 `neutral_atom` 根目录只保留 `session`、教程生成器、公开 `re-export` 和配置文件；真正的实现按架构层放进子包。这个布局的目的不是“看起来整齐”，而是让以后加硬件、算法、GUI 时能一眼看出应该放在哪里。

Package tree

```
Zou_lab_control/neutral_atom/
__init__.py          # public API: import
↳ Zou_lab_control.neutral_atom as na
session.py           # NeutralAtomSession / connect
notes.py             # this manual generator
configs/             # virtual / real device graph examples

core/
  analysis.py        # pure image/statistics helpers
  calibration.py     # TrapCalibration data record
  results.py         # ResultObject / CaptureResult /
  ↳ DetectionResult / ...
  utils.py           # small serialization/index helpers

devices/
  base.py            # BaseDevice / CameraDevice / SequencerDevice /
  ↳ TrapArrayDevice
  registry.py        # load_devices and config dependency resolution
  virtual.py         # VirtualTrapArray / VirtualCamera /
  ↳ VirtualSequencer
  qcmos.py           # real qCMOS adapter boundary
  sequencer.py       # VerilogSequencer adapter boundary

timing/
  sequence.py        # Pulse / PulseSequence / imaging_sequence
```

```

verilog.py          # edge table -> Verilog bundle

operations/
  calibration.py    # standalone sitemap / threshold algorithms
  detection.py      # standalone image -> DetectionResult

subsystems/
  readout.py        # exp.readout: sitemap / threshold / detect /
    ↪ fidelity scan
  timing.py         # exp.timing: configure / preflight /
    ↪ write_verilog

views/
  plots.py          # neutral atom objects -> frontend.plot
    ↪ adapters

```

2.2 依赖方向

最重要的规则是依赖只能往下走，不能绕回去。这样 notebook、GUI 和真实硬件都可以共享同一套底层对象。

Dependency direction

```

notebook / GUI
  |
  v
NeutralAtomSession
  |
  +--> devices/*      # hardware actions only
  +--> subsystems/*   # experiment-level organic actions
  +--> timing/*       # pulse and Verilog timing source of truth
  |
  v
operations/*          # standalone array algorithms
  |
  v
core/*                # data records, result objects, analysis helpers
  |
  v
views/* -> frontend    # plotting adapters only

```

`devices` 不应该 import `subsystems`，因为真实 hardware adapter 不应该知道实验动作。`operations` 不应该 import `session`，因为离线重分析不应该要求连接设备。`views` 只把 neutral-atom 数据转换成 `frontend.plot` 的输入，不应该控制 camera 或 timing。这个方向约束比文件名更重要：它防止以后重新长出一个什么都知道的“上帝对象”。

2.3 为什么 camera 是 device

`capture` 的物理语义是“camera 采图”。所以它属于 `CameraDevice`：真实 qCMOS 要 arm buffer、等 trigger、读 frame；virtual camera 要根据 trap array 渲染图像；两者都返回 image list。它们的共同 contract 是：

Python

```
class CameraDevice(BaseDevice):
    @property
    def exposure(self) -> float: ...

    def configure(self, *, exposure=None, **kwargs) -> None: ...

    def acquire(self, frames=1, *, sequence=None, sequencer=None,
        ↪ **kwargs) -> list[np.ndarray]: ...

    def capture(self, *, frames=1, exposure=None, sequence=None,
        ↪ display=True, **kwargs) -> CaptureResult:
        ...
```

`capture` 比 `acquire` 多做两件 notebook 友好的事：把最后一张 raw frame 交给 frontend 画出来，并把 raw images、sequence、plot handle 包成 `CaptureResult`。它永远不画 sitemap/site 圈，即使当前 session 已经做过 calibration。site overlay 只属于 `detect` 或显式的 calibration/readout 图；`capture` 不应该知道 site centers、threshold 或 virtual simulator state。

2.4 为什么 readout 是一个 subsystem

sitemap、threshold、detect、detection-time fidelity calibration 不是四个彼此无关的独立动作。它们共同依赖 camera readout 模型和同一份 `TrapCalibration`：

- `sitemap` 建立 site centers、grid ordering、ROI radius。
- `thresholds` 在这些 centers 上建立 per-site thresholds。
- `detect` 消费 centers 和 thresholds，输出 occupancy。
- `detection_time` 扫 exposure/readout time，仍然要在同一套 centers/ROI/count distribution 上评价 fidelity。

所以这些动作被放进一个 `ReadoutSubsystem`，入口是：

Python

```
exp.readout.sitemap(...)
exp.readout.thresholds(...)
exp.readout.detect(...)
exp.readout.detection_time(...)
exp.readout.save(...)
exp.readout.load(...)
exp.readout.clear()
```

这个 subsystem 是“实验层调度器”，不是算法仓库。它调用 `operations` 里的 standalone 函数，把当前 session 的 camera、sequence、defaults、calibration 和 history 串起来。这样在线实验和离线 array 重分析共用同一份算法。

2.5 session 的职责边界

`NeutralAtomSession` 是实验配置容器，不是所有动作的集合。它只持有：

- `devices`：实际 device graph。

- `sequence`: 当前 pulse sequence。
- `readout`、`timing`: 有机 subsystem。
- `_calibration`: 当前 readout state。
- `history`: 返回过的 result objects。

用户层保持短调用，但语义不混乱：

Python

```
exp = na.connect("virtual")
capture = exp.camera.capture()
sitemap = exp.readout.sitemap()
threshold = exp.readout.thresholds()
shot = exp.readout.detect()
scan = exp.readout.detection_time(times)
```

注意这里没有把 `capture`、`calibration`、`detect`、`characterization` 都挂成 session 顶层属性。那样要么让 session 变胖，要么把同一个 readout state 拆成几个假装独立的对象。

2.6 文件边界

core/analysis.py ROI counts、site finding、threshold、fidelity estimate。这些函数不认识 session，也不碰硬件。

core/calibration.py `TrapCalibration` 数据结构与保存读取。

core/results.py result object、task stop 契约和 notebook HTML summary。

devices/base.py `BaseDevice`、`CameraDevice`、`SequencerDevice`、`TrapArrayDevice`。这里定义换硬件时必须继承的接口。

devices/registry.py 把 JSON/dict 配置解析成 `DeviceSet`，支持 `"$device:name"` 依赖引用，并要求每个 device 继承对应 base class。

devices/virtual.py 离线 virtual trap array、camera、sequencer。

devices/qcmos.py 真实 qCMOS camera adapter 边界。

devices/sequencer.py FPGA/Verilog sequencer adapter 边界。

timing/sequence.py `PulseSequence`、pulse validation、pulse plot。

timing/verilog.py pulse edge table 到 Verilog bundle。

operations/calibration.py standalone sitemap 和 threshold calibration，从 image stack 到 `SitemapResult` / `ThresholdResult`。

operations/detection.py standalone 单张 image detect，从 image + calibration 到 `DetectionResult`。

views/plots.py neutral atom data 到 `frontend.plot` 的适配层。

subsystems/base.py `ExperimentSubsystem`，所有 exp-bound subsystem 的共同基类。

subsystems/readout.py `ReadoutSubsystem`，一个整体管理 sitemap、threshold、detect 和 readout-fidelity calibration。

subsystems/timing.py `TimingSubsystem`, 管理当前 sequence、preflight 和 Verilog。

session.py `NeutralAtomSession` 和 `connect`; 它绑定 device/subsystem, 不承载 standalone 算法和 result class。

2.7 device contract

真实硬件接入不是“刚好写一个同名方法”。camera adapter 应该继承 `CameraDevice`, 至少满足:

Python

```
class MyCamera(na.CameraDevice):
    @property
    def exposure(self) -> float: ...

    def configure(self, *, exposure=None, **kwargs) -> None: ...

    def acquire(self, frames=1, *, sequence=None, sequencer=None,
        ↪ **kwargs) -> list[np.ndarray]: ...
```

`acquire` 的职责是 arm camera、让 sequencer prepare/fire (如果需要)、读出 frame, 并返回 numpy image list。它不应该做 site threshold、fidelity 或保存图; 这些属于 `operations / result object`。sequencer adapter 对应 `SequencerDevice`, 必须提供 `prepare(sequence)` 和 `fire(sequence)`。如果一个 camera 类没有实现这些 abstract method, Python 会禁止实例化; 如果 JSON 配置里的 `camera` 没有继承 `CameraDevice`, `load_devices` 会直接报错。

2.8 session 里面有什么

Python

```
exp = na.connect("virtual")

exp.devices      # raw DeviceSet
exp.camera       # actual CameraDevice, e.g. VirtualCamera or QCMOSCamera
exp.readout      # sitemap / threshold / detect / readout-fidelity
↪ subsystem
exp.timing       # pulse / preflight / verilog subsystem
exp.readout.current # current TrapCalibration or None
exp.history      # returned result objects
```

2.9 standalone 算法入口

并不是所有事情都需要 session。三类函数可以直接处理 array:

Python

```
sitemap = na.calibrate_sitemap_from_images(images, grid_shape=(5, 7))
threshold = na.calibrate_threshold_from_images(images,
    ↪ sitemap.calibration)
shot = na.detect_image(image, threshold.calibration)
```

这条 standalone 路径对真实实验很关键。很多时候你已经有 raw image stack, 只想重调 ROI radius、ordering 或 threshold; 这时不应该重新连接 camera, 也不应该让分析函数依赖 GUI 状态。

2.10 Result object 和 summary()

subsystem/device 调用的返回值统一叫 result object, 例如 `CaptureResult`、`SitemapResult`、`ThresholdResult`、`DetectionResult`、`DetectionTimeScanResult` 和 `PreflightReport`。它们不是只为打印好看而存在, 而是 notebook、GUI 和自动日志之间的稳定边界。

每个 result object 都保留三类信息:

- raw 或近 raw 数据, 例如 `CaptureResult.images`、`SitemapResult.average_image`、`ThresholdResult.counts`、`DetectionResult.image`、`DetectionResult.occupied`。
- 当前 plot handle, 例如 `result.plot`, 用于 notebook 里继续调 axis、保存图, 或让 GUI panel 接管显示。
- 一个小而稳定的 `summary()` dict, 用于快速查看、保存状态、测试断言和未来 GUI 的状态栏。

`summary()` 的设计原则是 “只放轻量、JSON-friendly、不会巨大展开的关键信息”。它不替代 raw data: 真正要做分析时应该读 `result.images`、`result.counts`、`result.occupied` 或 `result.calibration`。Jupyter 里直接显示 result object 时, `_repr_html_` 也是调用 `summary()` 生成一小块 HTML, 所以 notebook 不会因为一个 image stack 或 counts matrix 被整个刷出来。

Python

```
capture = exp.camera.capture()
capture.summary()
# {'frames': 1, 'image_shape': [96, 128], 'sequence': 'imaging', ...}

shot = exp.readout.detect()
shot.summary()
# {'loaded_atoms': 18, 'occupied_indices': [0, 3, 4, ...]}
```

长期架构里, `summary()` 也是 GUI 和实验调度可以依赖的最小状态接口: 按钮回调拿到 result 后可以立刻更新状态栏、写一行 JSON log、或者在失败时给出可读诊断, 而不会把大型 numpy array 或 Matplotlib artist 混进控制层。

scan-like task 的统一契约

以后所有 scan / measurement / async task result 都应该继承同一类契约, 而不是各自临时发明接口。公共结构是:

- `result.stop()`: task 对外唯一 stop, 停止 measurement worker/source 和 attached frontend refresh。
- `result.measurement`: 采集/执行任务本身, 拥有底层 lifecycle 和状态, 例如 `points\_done`、`running`、`done`。
- `result.plot`: frontend plot 对象, 只负责显示和交互。
- `result.data\_figure`: frontend 后处理对象, 负责 fitting、unit conversion、save 等。
- result 顶层只保存实验语义数据, 例如 `times`、`fidelities`、`thresholds`、`calibration`; 不重复包装 `DataFigure.decay()` 这类已经存在的方法。

这样 GUI 以后对外只需要把 Stop 按钮连到 `result.stop()`; 需要 debug 或高级接管时, 仍然能看到 `measurement`、`plot` 和 `data\_figure` 三个部件。

3

Frontend 更新

3.1 统一 plot 契约

frontend 的公开契约仍然是 `data_x: (N, coord_dim)` 和 `data_y: (N, channel_dim)`。已有数组和 live plot 本质上是同一个 plot: 静态时直接画当前 array; live 时 acquisition worker 写共享 array, frontend timer 只刷新 artist。

3.2 标题

所有 plot 现在都接受 `title=...`。标题字号等于 axis label 字号, 不会变成 notebook 里的大标题; 有标题时默认增加 top margin, 避免被图边界截断。

3.3 Pulse sequence 图

新增 `zf.plot(sequence, kind="pulse")`。它用实心低饱和色块表示 TTL pulse, y 轴 label 与该 channel 色块同色, 并缩小 tick label 来容纳至少 10 行 channel。x 轴按总时长自动选择 `ns/us/ms/s`, 例如微秒级 sequence 会显示 `Time (us)`, 避免时序图上全是很长的科学计数法秒数。off baseline 和 on block 使用完全相同的颜色和不透明度, 只是几何形状不同: baseline 是同色细线, on interval 是从 baseline 向上长出的实心块。这样 pulse 看起来是一条同色信号轨迹, 而不是两套叠加图层。pulse plot 用来检查时间结构, 不在色块内部强行塞文字, 避免短 pulse 上 label 被截断。

3.4 2D 图保持 square 公开接口

`frontend.plot(..., kind="2d")` 对外始终使用 square view。内部 `Live2DDis` 仍保留非 square 能力给开发调试, 但 public API 不允许 `square=False`。这样 colormap 主图、distribution axis 和 colorbar 的视觉对齐更稳定。

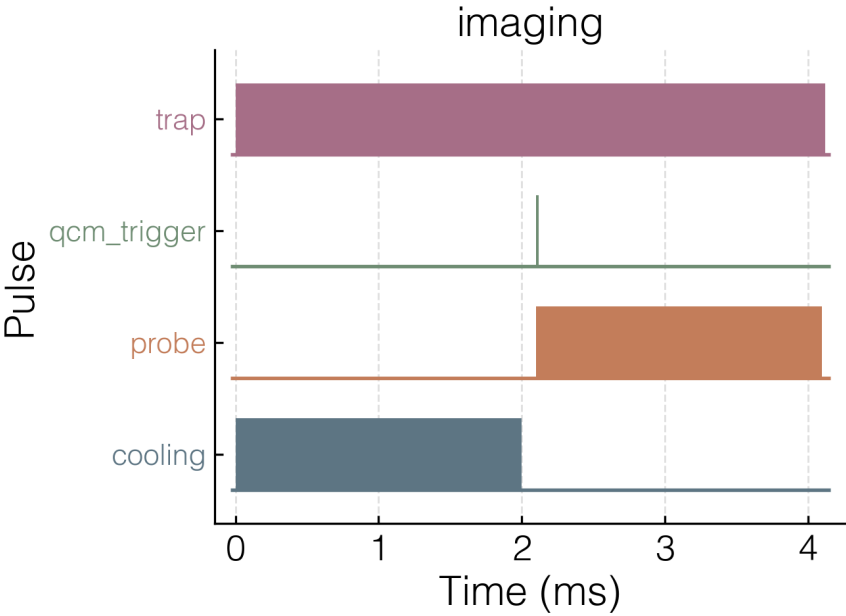


Figure 3.1: 当前 imaging sequence 的 pulse plot.

4

Device 配置与 virtual camera

4.1 最小 device graph

JSON

```
{
  "trap_array": {"type": "VirtualTrapArray"},
  "camera": {
    "type": "VirtualCamera",
    "params": {"trap_array": "$device:trap_array"}
  },
  "sequencer": {"type": "VirtualSequencer"}
}
```

`load_devices` 会按依赖关系构造 device 并校验 base-class contract。真实硬件配置只需要把 camera/sequencer class 换掉；只要新 class 继承并实现 `CameraDevice` / `SequencerDevice`, subsystem 层仍然调用 `camera.acquire(...)` 和 `sequencer.fire(...)`。

4.2 qCMOS-like simulation

virtual camera 不再用任意 counts 噪声。它参考 C15550-22UP instruction manual 中的典型量级：

- dark/baseline offset 约 200 counts。
- conversion factor 0.107 electrons/count。
- standard scan readout noise 0.43 electrons RMS。
- ultra quiet scan readout noise 手册给出 0.30 electrons RMS, 当前默认先模拟 standard scan。
- -35 °C dark current 量级 0.006 electrons/pixel/s。

模拟流程是：先在电子数空间生成 background、dark current 和 Gaussian atom spot；对期望电子数做 Poisson shot noise；再用 conversion factor 转回 counts，加 baseline offset 和 readout noise。这样短 exposure 的分布会自然变差，长 exposure 的分布会变好；readout calibration 只能从

这些模拟相机图像里估计结果，不能从 simulator 的内部状态直接取答案。

为什么默认不让 lifetime 影响第一版 scan

真实实验里 detection time 过长确实可能造成 atom loss。但第一版要先校准 camera SNR 对 fidelity 的影响，所以 virtual 默认 detection lifetime 设为很长，避免 fidelity curve 被模拟寿命伪影拖成“时间越长越差”。以后可以把 loss model 做成 scan strategy 的显式参数。

5

Timing、preflight 和 Verilog

5.1 PulseSequence 的作用

PulseSequence 保存物理时间:

Python

```
seq = na.imaging_sequence(exposure=2e-3, load=True)
seq = seq.delay("qcm_trigger", 4e-9)
```

默认 imaging sequence 包含 `trap`、可选 `cooling`、`probe` 和 `qcm_trigger`。sequence 本身不接触 camera，也不直接写 Verilog；它只是可验证的 timing source of truth。

5.2 preflight 检查

`exp.timing.preflight()` 检查:

- pulse 是否有负时间或非正 duration。
- 同一 channel 是否存在 overlap。
- pulse 是否短到低于一个 FPGA clock tick。
- sequence channel 是否在 sequencer channel list 中。
- 可选生成 Verilog edge table 摘要。

5.3 Verilog 生成

`generate_verilog` 把 pulse 编译成绝对 tick 和 channel mask。生成 module 有 `clk/reset/start/running/done` 和各 channel 输出。当前没有复刻完整 address-switch register/VIO 体系，这是有意收缩：第一版先把 pulse timing 从手写 cumulative if 变成 Python 可检查的 edge table。

6

Camera capture

6.1 推荐调用

Python

```
capture = exp.camera.capture()
```

这一步调用当前 `exp.sequence`。对 virtual camera, 它直接渲染模拟图; 对真实 camera, 应该由 `QCMOSCamera.acquire` 完成 arm/trigger/readout/release。camera device 只负责采图, 不负责 threshold 或 detect。

capture 必须是 raw image

`exp.camera.capture()` 的图里不允许自动叠加 sitemap 圈。哪怕当前 session 已经有 `TrapCalibration`, capture 仍然只显示相机 raw data。需要检查 site map 对齐或 occupied atoms 时, 使用 `exp.readout.sitemap()`、`exp.readout.detect()` 或 `DetectionResult.plot\_occupancy()`。

6.2 CaptureResult

`CaptureResult` 保存 `images`、`sequence` 和 `plot`。如果以后要保存 raw image stack, 应在 result object 上加 `save_npz`, 不要让 camera 在 `acquire` 内部偷偷写文件。

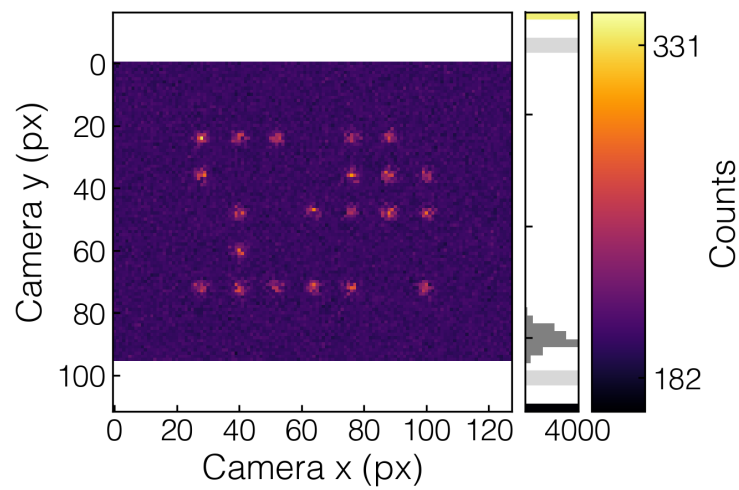


Figure 6.1: 一次 virtual qCMOS capture.

7

Sitemap calibration

7.1 推荐调用

Python

```
sitemap = exp.readout.sitemap(frames=12, roi_radius=1)
```

sitemap calibration 拍多帧 sitemap 图，平均后找 bright local maxima，再按 ordering 排成稳定 site index。它只回答 “site 在哪里”，不回答 “occupied threshold 是多少”。

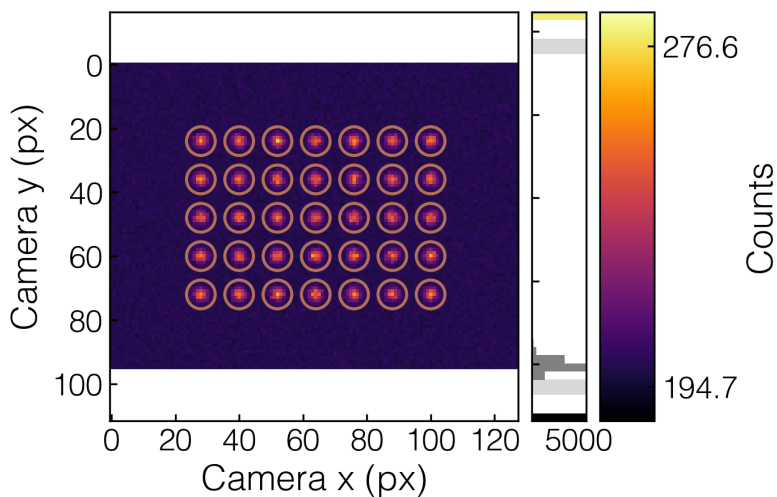


Figure 7.1: 平均 sitemap 图和自动找到的 site centers。

7.2 常见错误

- center 整体偏移：优先检查 camera ROI、image orientation、ordering。
- 少找 site：增加 sitemap frames，降低 threshold 或检查是否所有 site 被点亮。

-
- site index 顺序不对: 改 `ordering`, 不要在 detect 里临时重排。

8

Threshold calibration

8.1 推荐调用

Python

```
threshold = exp.readout.thresholds(frames=80, site=0)
```

threshold calibration 在已知 centers 上重复拍图，得到 `counts: shots x sites`。每个 site 用 Otsu threshold 做初始阈值，显示时用 frontend histogram。threshold 线可拖动，右上角显示当前 threshold、左右比例和双峰 Gaussian 模型 fidelity。

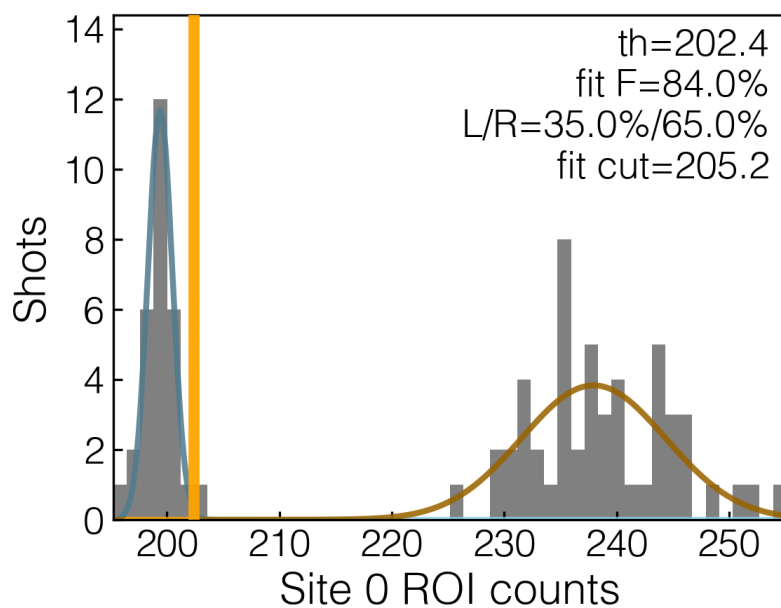


Figure 8.1: site threshold histogram.

8.2 为什么不能用 simulator truth 标定

真实实验没有逐 shot ground truth。virtual device 内部当然有用于生成图像的 hidden occupancy state, 但这个状态不能出现在 `CaptureResult`、`DetectionResult`、session status 或 fidelity API 里。当前 readout-fidelity calibration 的 fidelity 来自可观测图像分布: threshold 两侧的 Gaussian split、左右比例和 reference exposure。这样同一套逻辑可以迁移到真实 qCMOS。

9

Detect occupancy

9.1 推荐调用

Python

```
shot = exp.readout.detect()  
occupied = shot.occupied
```

`detect` 要求已经有 `sitemap` 和 `threshold`。它拍一张 raw image, 提取每个 site 的 ROI count, 再和 per-site threshold 比较。输出 `DetectionResult.occupied` 是 boolean array, 可直接传给 rearrangement planner 或统计函数。

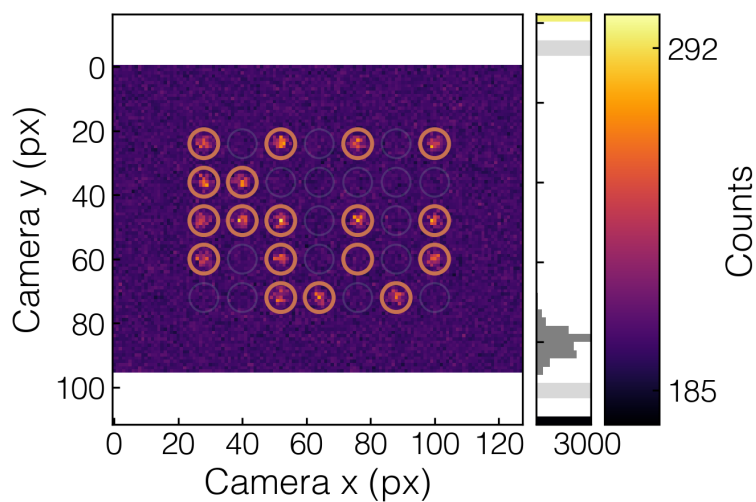


Figure 9.1: raw image 上叠加 sitemap faint rings 和 occupied rings.

9.2 图像设计

detect 图不再只画一个抽象 occupancy heatmap。背景是 raw camera data；每个 sitemap site 有较大的浅青灰色 ring，尺寸与 sitemap-found overlay 保持一致；只有判断为 occupied 的 site 再叠加较细的橙色 ring。这样可以同时检查 raw signal、site map 对齐和最终 binary classification。

10

Detection-time fidelity scan

10.1 推荐调用

Python

```
times = np.linspace(0.2e-3, 10e-3, 100)
scan = exp.readout.detection_time(times, shots=30, live=True)
```

`detection_time` 默认 `live=True`。cell 会很快返回, `scan` 在 frontend-owned worker 里继续跑; worker 负责采集并写共享 `data_y`, plot timer 只刷新图。这样采集不会阻塞 Jupyter UI, 也给未来 PyQt 主线程绘图留出边界。教程里保留 live scan: 等图跑完或需要提前停止时, 先运行 `scan.stop()`, 再在后面的 cell 里做 fit。

10.2 停止 live scan: 一个对外 stop

长扫描对外只提供一个 stop, 完成后或需要提前中断时运行:

Python

```
scan.stop()
scan.summary()
```

`scan.stop()` 来自 task base class, 它转发到 frontend `RunSession.stop()`。这个 session 会设置 stop event、调用 source/device 的 `stop()` (如果存在)、停止 plot refresh timer, 并做一次 final refresh, 让已采到的数据留在图和 `scan.fidelities` 里。真实硬件接入时, device/source 的 `stop()` 必须能打断阻塞采集, 否则 Python 线程只能在当前 shot/acquire 返回后自然退出。

10.3 用 `scan.data_figure` 做 decay fit

拟合前要保证数据已经采完。交互式实验里可以等 live 图跑完, 再先执行上一节的 `scan.stop()` 做 final refresh; 然后在后面的 cell 里执行拟合。

scan result 不应该复制 `DataFigure` 的方法。它只暴露组成部分: `scan.measurement` 管 lifecycle, `scan.plot` 管显示, `scan.data\_figure` 管 fitting/saving/post-processing。因此

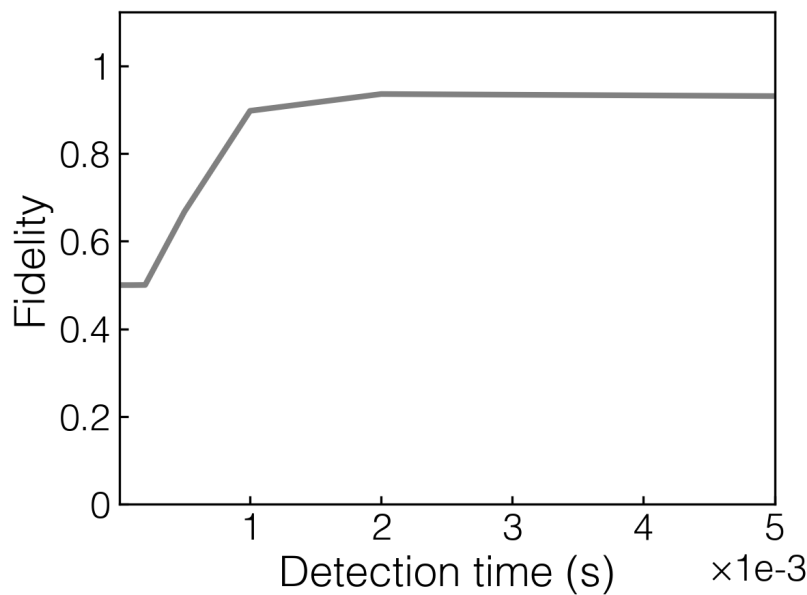


Figure 10.1: detection time 与 reference-calibrated model fidelity。

decay fit 直接调用已有的 frontend fitting 栈:

Python

```
fit_result, popt = scan.data_figure.decay()  
scan.summary(), fit_result, popt
```

10.4 reference exposure 的角色

`detection_time` 先拍 long-exposure reference images, 用真实实验也能得到的图像分布建立参考。然后每个 detection time 都独立拍 `shots` 张, 估计该 exposure 下的 threshold 和 Gaussian split fidelity。

当前 result 保存:

- `scan.fidelities`: 画在图上的 fidelity。
- `scan.thresholds`: 每个 detection time 的自动 threshold。
- `scan.model_fidelities`: 每个 time 的 Gaussian model fidelity。
- `scan.reference_counts`: long-exposure reference ROI counts。
- `scan.reference_threshold` 与 `scan.reference_fidelity`。

后续更严格的策略

真实实验如果需要更严谨的 detection fidelity, 可以扩展成 paired reference: 每次短 exposure detect 后加一张 long exposure reference, 或在同一 loading 条件下做 before/after reference。这个策略应该加在 `ReadoutSubsystem` 或更高层 experiment protocol, 不应该塞进 camera, 也不是让 virtual truth 混进 fidelity API。

11

保存输出

11.1 calibration、status、Verilog

Python

```
cal_path = exp.readout.save("results/calibration.json")
status_path = exp.save_status("results/status.json")
verilog_path = exp.timing.write_verilog("generated_sequences")
```

`save_status` 会保存 device snapshot、current sequence、calibration 和 history length。Verilog bundle 会写 `.v` 和 manifest, manifest 里包含 clock、channels、ticks、masks 和 source hash。

12

以后加功能应该放哪里

12.1 加真实 camera

真实 camera adapter 放在 `devices/`, 继承 `CameraDevice`。它应该只处理硬件生命周期和 image acquisition:

- `open/close/stop` 管资源。
- `configure` 管 exposure、ROI、readout mode 等稳定参数。
- `acquire` 管 arm、trigger、wait frame、readout。
- `snapshot` 返回轻量 metadata, 给 status log 和 GUI 状态栏使用。

不要在 camera adapter 里做 threshold、site finding、plot styling、保存实验结果。这些分别属于 `operations/`、`views/` 和 result object。

12.2 加新的图像算法

如果算法只需要 image/counts/calibration, 就放在 `operations/` 或 `core/analysis.py`。判断标准是: 这个函数能不能在没有 camera、没有 session、没有 GUI 的 Python 脚本里跑。如果能, 就不要放进 subsystem。subsystem 只是把当前实验状态喂给算法。

12.3 加新的实验动作

如果动作需要多个 device、当前 sequence、defaults、calibration 或 history, 就放在 `subsystems/`。例如 readout 相关动作放 `ReadoutSubsystem`; 以后 rearrangement 可以是 `RearrangementSubsystem`, 它消费 `DetectionResult.occupied`, 调用 AOD/PXIE device, 输出 move plan 和执行 result。

12.4 加新的硬件角色

如果是新的硬件类型, 先在 `devices/base.py` 定义一个明确 base class, 例如 `AODDevice` 或 `MicrowaveDevice`, 再让 virtual/real implementation 继承它。这样错误会在类实例化或 `load_devices` 阶段暴露, 而不是实验跑到一半才发现少了方法。

12.5 加 GUI

GUI 不应该重新实现实验逻辑。GUI panel 持有一个 `NeutralAtomSession`，按钮只调用 `exp.camera.capture()`、`exp.readout.detect()`、`exp.timing.preflight()` 这类现成接口。画图层继续接受 result 的 `plot` 或 `data\_figure`。如果 PyQt 要求绘图在主线程，采集线程只负责写 queue/shared array，UI timer 只负责读数据并刷新 frontend/Qt artist。

13

下一步扩展计划

13.1 真实 qCMOS

- 增加 DCAM error report 和 timeout diagnostics。
- 明确 external trigger mode、trigger polarity、readout direction 和 exposure timing。
- 对 ROI/subarray 做硬件约束检查。
- 保存每次 capture 的 exposure/readout metadata。

13.2 sequencer 和 address-switch

- 当前 `timing/verilog.py` 已经能从 pulse 生成 edge table。
- 下一步把 address-switch register/VIO/download/start handshake 包成 `VerilogSequencer` 的真实实现。
- notebook 仍然只调用 `exp.timing.write_verilog()`、`exp.timing.preflight()` 和 camera/readout subsystem。

13.3 rearrangement 和 GUI

- rearrangement planner 应消费 `DetectionResult.occupied`，输出 move plan。
- AOD/PXIE device 应该是独立 device class，不应该塞进 detector。
- GUI panel 持有 session，按钮调用 device/subsystem method；plot surface 继续用 frontend data contract。
- PyQt 画图必须在主线程时，采集 worker 只通过 thread-safe queue/shared array 交数据，UI timer 只读数据并刷新。